

# Contents


**1 The Multivariate Normal Distribution** *3*

1.1 The Covariance Matrix *5*

1.1.1 Computing Mean and Covariance in Python *6*

1.2 Multivariate Normal Probability Density *6*

1.2.1 Multivariate Normal Probability Density in Python *8*

**2 Numerical Double Integration** *9*

2.1 Riemann Sums on 2-Dimensional Domains *10*

2.2 Numerical Integration in Python *12*

**3 Bayesian Classifiers** *15*

3.1 The Prior *16*

3.2 Naive Bayes *17*

3.3 Using the multivariate Gaussian distribution *19*

**A Answers to Exercises** *23*

**Index** 25

CHAPTER 1

# The Multivariate Normal Distribution

Let's say that you are gathering statistics on a species of snail. You have found 89 snails. Every snail has been weighted and had its shell measured. You have created a table like this:

| Weight | Diameter |
|---|---|
| 4.4769 grams | 2.2692 cm |
| 5.4755 grams | 2.1973 cm |
| 4.1183 grams | 2.52928 cm |
| ... | ... |
| 3.0522 grams | 1.7822 cm |

You have been told that for any particular species of snail, the weight and shell diameter are typically normally distributed. So, you compute the mean of each:

$$\bar{x} = \frac{\sum_{i=1}^{n} x_i}{n}$$

Your snails have a mean mass off 4.25 grams and a mean shell diameter of 2.35 centimeters. Now you know where the center of each normal distribution will be.

To know how wide each normal distribution is, you need to compute the variance:

$$\sigma^2 = \frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n}$$

(Reminder: The standard deviation is the square root of $\sigma^2$)

Plotting this data is shown in Figure 1.1.

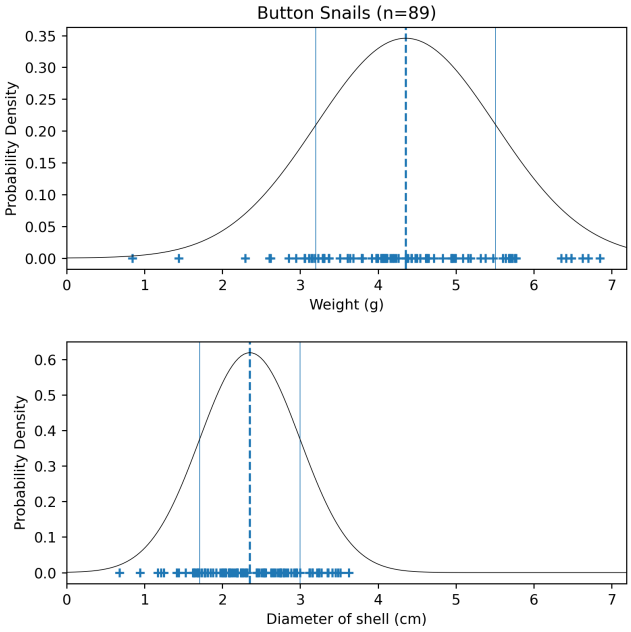



Figure 1.1: Probability density functions of weight and diameter.

Nice! However, then you think: ”Hmm. Maybe the mass and the diameter of the shell are related. It seems like a heavy snail might tend to have a larger shell.” So, you make a scatter plot:

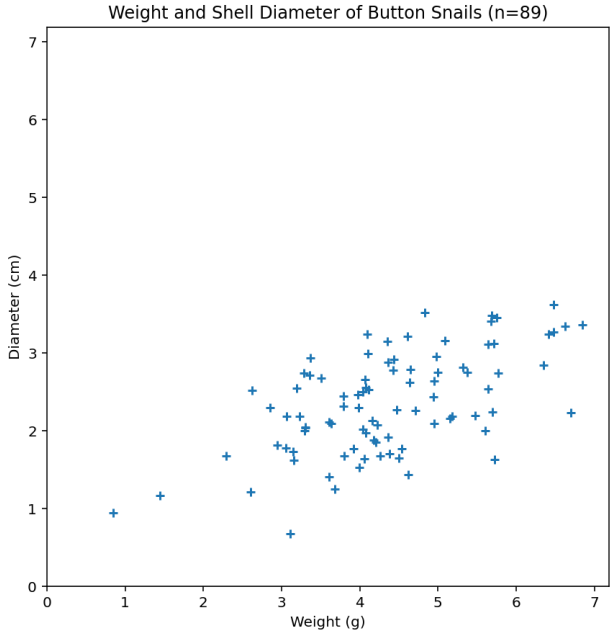



Figure 1.2: A scatter plot with weight of the snail horizontally and diameter of the shell on the y-axis.

Sure enough, there is some correlation between the mass of the snail and the diameter of its shell.

There is a form of the normal distribution that can deal with multiple variables and their correlations. This is known as the *multivariate normal* or the *Gaussian distribution*.

In a multivariate normal distribution, the mean (often denoted $\mu$) is a vector containing the mean of every variable. For your snails, $\mu = [4.25, 2.35]$.

Just as in the single-variable normal distribution, measurements near the mean are what you are most likely to observe. In a single-variable normal distribution, the standard deviation tells us how fast that likelihood falls off as we move away from the mean. In multivariate normal distributions, we need something a little more expressive: a matrix.

## 1.1 The Covariance Matrix

We can think of the data as a list of vectors. For example, if we have $n$ snails and $d$ properties that we have measured, we would get a list like this:

$$\begin{bmatrix} x_{1,1} & x_{1,2} & \dots & x_{1,d} \\ x_{2,1} & x_{2,2} & \dots & x_{2,d} \\ \dots & \dots & & \dots \\ x_{n,1} & x_{n,2} & \dots & x_{b,d} \end{bmatrix}$$

Each row represents one snail. Each column represents one property.

Note that to make $\mu$ (the mean vector), you just take the mean of each column of this data matrix. (For convenience, we will use $\mu_i$ to refer to the mean of column $i$.)

We usually use $\Sigma$ (the uppercase Greek sigma) to represent a covariance matrix. $\Sigma$ is a $d \times d$ matrix. The entries on the diagonal of $\Sigma$ are just the variance of each property. So, we can calculate the entry at row $j$, column $j$ like this:

$$\Sigma_{j,j} = \frac{\sum_{i=1}^{n}(x_{i,j} - \mu_j)^2}{n}$$

(Yes, we are using $\Sigma$ to represent both summation and the covariance matrix here. It can be confusing, but you will be able to tell from context how it is being used.)

The other entries, $\Sigma_{j,k}$ where $j \neq k$, are the *covariance* between property $j$ and property $k$. If the covariance is a positive number, property $j$ and property $k$ are correlated. For example, in your snails, weight and diameter have a positive covariance: If the snail is

heavier than average, it tends to have a diameter that is larger than average.

If the covariance is a negative number, when property $j$ is greater than average, property $k$ tends to be less than average. For example, the number of times a restaurant mops the floor in a week has a negative covariance with how much bacteria lives on the floor.

How do we compute the covariance?

$$\Sigma_{j,k} = \frac{\sum_{i=1}^{n}(x_{i,j} - \mu_j)(x_{i,k} - \mu_k)}{n}$$

Note that $\Sigma_{j,k} = \Sigma_{k,j}$, so the matrix is symmetric.

### 1.1.1 Computing Mean and Covariance in Python

If you have a numpy array where each row represents one sample and each column represents one property, it is really easy to compute $\mu$ and $\Sigma$:

```
# Read in the data matrix, each row represents one snail
snails = ...

# Compute mu
mean_vector = snails.mean(axis=0)
print(f"Mean = {mean_vector}")

# Compute Sigma
covariance_matrix = np.cov(snails, rowvar=False)
print(f"Covariance = {covariance_matrix}")
```

## 1.2 Multivariate Normal Probability Density

Now that we have good estimates of $\mu$ and $\Sigma$, how can we compute the probability density?

When you were working with just one variable, $x$, you computed the probability density using the mean $\mu$ and the variance $\sigma^2$ like this:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

The probability density function of a $d$-dimensional multivariate normal distribution with a mean of $\mu$ and a covariance matrix of $\Sigma$ is given by:

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x}-\boldsymbol{\mu})\right)$$

If we draw lines to show where $\boldsymbol{\mu}$ is and contours to show lines of equal probability density, we get a plot shown in Figure 1.3.

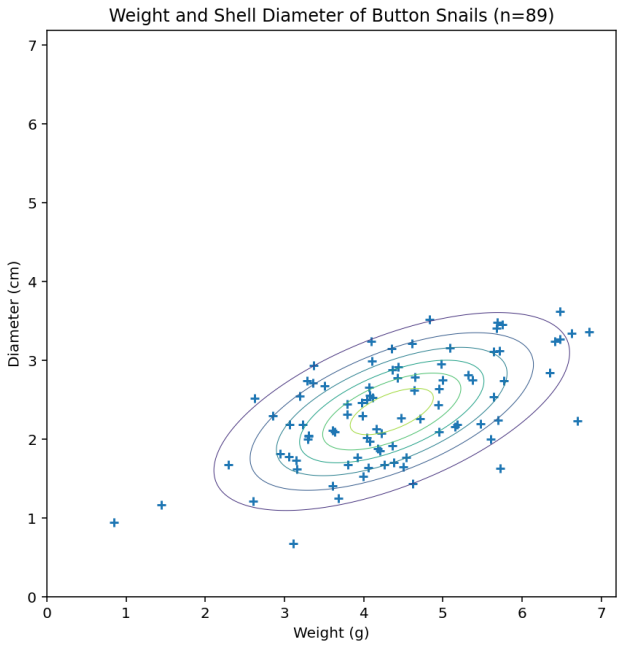



Figure 1.3: Contour plot of the snail data.

Or, we can do a 3D plot, as shown in Figure 1.4.

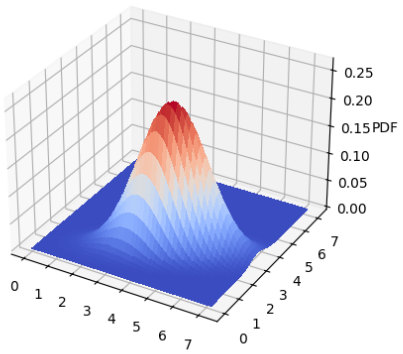



Figure 1.4: 3D plot of the probability density function.

A probability density always integrates to 1.0, so the volume under the surface must be 1.0.

### 1.2.1 Multivariate Normal Probability Density in Python

The scipy library has a class that represents the multivariate normal. Here is how you could compute the probability density for a particular snail:

```
import numpy as np
from scipy.stats import multivariate_normal

...Compute mean_vector and covariance_matrix...

# Get the probability density at [3 grams, 2 cm]
x = np.array([3.0, 2.0])
pd = multivariate_normal.pdf(x, mean=mean_vector, cov=covariance_matrix)
print(f"The probability density at {x} is {pd}")
```

Alternatively, maybe you would like to generate weights and diameters for a fictional population of 700 snails:

```
new_snails = multivariate_normal.rvs(mean=mean_vector, cov=covariance_matrix,
↪  size=700)
print(f"My fictional snails:\n{new_snails}")
```

# Chapter 2

# Numerical Double Integration

In an earlier chapter, we gave an example of the multivariate normal distribution: the mass and shell diameter of a population of snails.

Starting with a table of measurements, we estimated that the mean vector $\mu$ was $\begin{bmatrix}4.25\text{g} & 2.35\text{cm}\end{bmatrix}$.

We then computed the covariance matrix:

$$\Sigma = \begin{bmatrix}1.33 & 0.443 \\ 0.443 & 0.416\end{bmatrix}$$

Plotting the data points, the mean, and the equal-density contours looked like this:

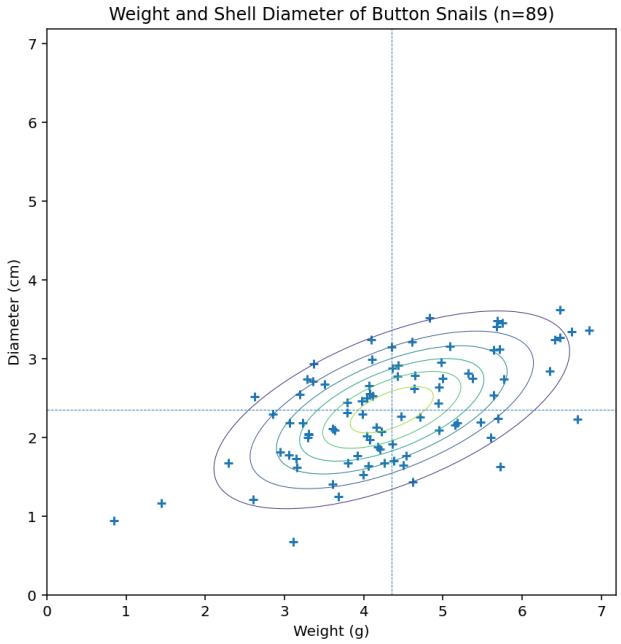



Figure 2.1: Equal density contours plotted with the snail data.

We have a great formula for computing the probability density for any mass/diameter combination:

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x}-\boldsymbol{\mu})\right)$$

Can you answer the following question? "If I pick a random snail off the floor of the ocean, what is the probability that its mass is between 3 and 4 grams and its diameter is between 1.5 and 2.0 centimeters?"

If we think of the probability density as a surface, this question is really "What is the volume under the surface in the rectangular patch $3 <= x_1 <= 4$ and $1.5 <= x_2 <= 2.0$?" Which you should recognize as a double integration problem:

$$P = \int_{1.5}^{2.0} \int_{3}^{4} \frac{1}{\sqrt{(2\pi)^d |\Sigma|}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x}-\boldsymbol{\mu})\right) dx_1 dx_2$$

Sadly, however, no one has ever been able to find the antiderivative of the multivariable probability density function, so no one can solve this problem.

Instead, we use Riemann sums to find an approximate solution. This is known as *numerical integration*.

(After spending so much time learning the techniques for integration, it may be disappointing to hear this: For a lot of real-world problems, there is no way to find an antiderivative, so we end up doing numerical integration much more often than most people realize.)

## 2.1 Riemann Sums on 2-Dimensional Domains

When doing Riemann sums on function that takes a single real number, you summed the area of the rectangles under the function to approximate the integral:

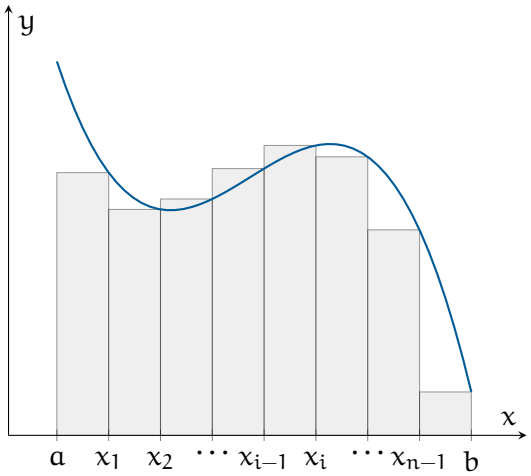



Figure 2.2: A representative function divided into $n$ rectangles of equal width, with rectangle height determined by the right endpoint of the subinterval

Here you are finding the volume under a function that takes two variables (the probability density is based on the mass and diameter of the shell).

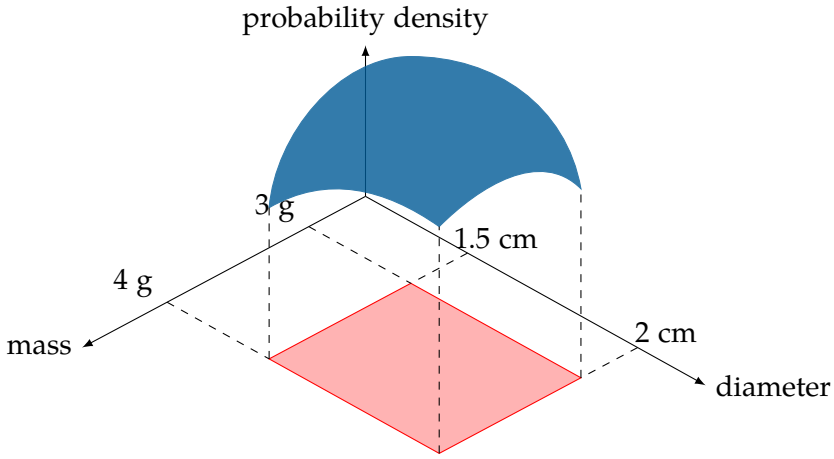



Figure 2.3: The probability is the volume under a surface for a region

To do the Riemann sum, we can break the range of the mass into $n_1$ equally sized intervals and the range of the diameter into $n_2$ equally sized intervals. For the diagram, we will just break each range into three, but you will get more accurate estimate as the intervals get smaller.

Now you will be calculating the volume of rectangular solids and summing up those volumes. Here are a few of the rectangular volumes:

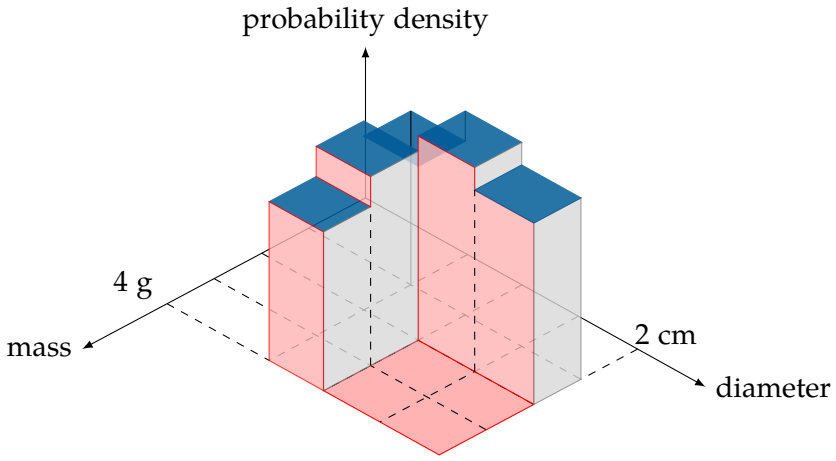



Figure 2.4: Riemann Sum

## 2.2 Numerical Integration in Python

Here's the code:

```
import numpy as np
from scipy.stats import multivariate_normal

# pip3 install scipy
weight_lower_limit = 3.0
weight_upper_limit = 4.0
weight_slices = 100

diameter_lower_limit = 1.5
diameter_upper_limit = 2.0
diameter_slices = 100

# What's the average weight and diameter
mean_vector = np.array([4.35559489,2.3526593 ])
print(f"Mean [weight, diameter] = {mean_vector}")

# Do heavier snails tend to have bigger shells?
covariance_matrix = np.array([[1.33099714,0.44309754],
 [0.44309754,0.41603925]])
print(f"Covariance = \n{covariance_matrix}")

# Create a multivariate normal distribution
rv = multivariate_normal(mean_vector, covariance_matrix)

delta_weight = (weight_upper_limit - weight_lower_limit) / weight_slices
delta_diameter = (diameter_upper_limit - diameter_lower_limit) / diameter_slices

sum = 0.0
```

```
# Step though each different weight
for i in range(weight_slices):
    # What is the weight in the middle of this slice?
    current_weight = weight_lower_limit + (i + 0.5) * delta_weight

    for j in range(diameter_slices):
        # What is the diameter in the middle of this slice?
        current_diameter = diameter_lower_limit + (j + 0.5) * delta_diameter

        # What is the probability density there?
        prob_density = rv.pdf((current_weight, current_diameter))

        # What is the volume under that for this tiny square
        sum += prob_density * delta_weight * delta_diameter

print(
    f"\nThe probability that the weight is between {weight_lower_limit} and
    ↪ {weight_upper_limit}"
)
print(
    f"and that the diameter is between {diameter_lower_limit} and
    ↪ {diameter_upper_limit}"
)
print(f"is about {sum * 100.0:.4f}%")
```

This should get you the following output:

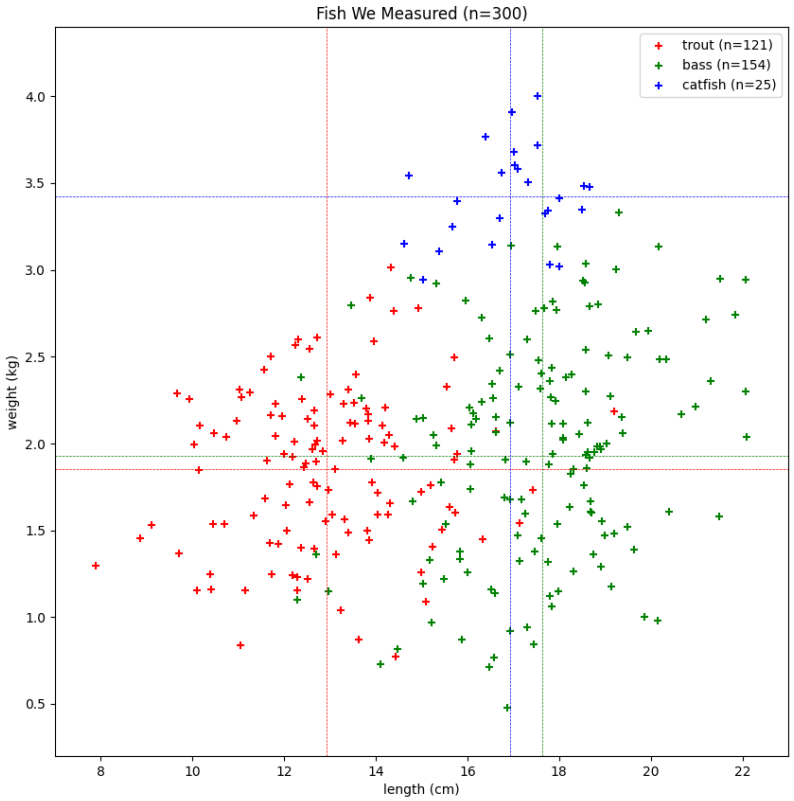
```
> python3 num_integration.py
Mean [weight, diameter] = [4.35559489 2.3526593 ]
Covariance =
[[1.33099714 0.44309754]
 [0.44309754 0.41603925]]

The probability that the weight is between 3.0 and 4.0
and that the diameter is between 1.5 and 2.0
is about 7.8316%
```

# Chapter 3

# Bayesian Classifiers

You drop a net in several places in a lake. You measure, weigh, and identify every fish to be one of the following: `trout`, `bass`, or `catfish`. Here is a scatter plot of what you find:




(Data: `make_data.py`, Graph: `1_scatter_fish.py`)

The vertical and horizontal lines represent the mean length and width (respectively) of each type of fish.

You are writing a system that someone on the lake could use to identify their fish.

## 3.1 The Prior

Even before the person measures and weighs the fish, they can make a decent guess at what any randomly selected fish is:

| Fish | Count | Percent |
|---|---|---|
| trout | 121/300 | 40.3% |
| bass | 154/300 | 51.3% |
| catfish | 25/300 | 8.3% |

If you know nothing about the fish, there is a 51.3% chance it is a bass. The probability without any observations is known as the *prior*:

$$\mathrm{P}(\text{fish} = \text{trout}) = .403$$
$$\mathrm{P}(\text{fish} = \text{bass}) = .513$$
$$\mathrm{P}(\text{fish} = \text{catfish}) = .083$$

However, now the user makes an observation: The fish is 16 inches long and weighs 2.1 kg. Can we update our beliefs based on this? Sounds like a job for conditional probability.

By the definition of conditional probability:

$$\mathrm{P}(\text{fish} = \text{trout} | \text{length} = 16, \text{weight} = 2.1) = \frac{\mathrm{p}(\text{fish} = \text{trout}, \text{length} = 16, \text{weight} = 2.1)}{\mathrm{p}(\text{length} = 16, \text{weight} = 2.1)}$$

At this rate, we are going to run out of space, so we will use $\mathrm{F}$ to represent fish, $\mathrm{L}$ to represent length, and $W$ to represent weight.

We can calculate the denominator by summing up all the possibilities. Now, the right-hand side is:

$$\frac{\mathrm{p}(\mathrm{F}{=}\text{trout}, \mathrm{L}{=}16, \mathrm{W}{=}2.1)}{\mathrm{p}(\mathrm{F}{=}\text{trout}, \mathrm{L}{=}16, \mathrm{W}{=}2.1) + \mathrm{p}(\mathrm{F}{=}\text{bass}, \mathrm{L}{=}16, \mathrm{W}{=}2.1) + \mathrm{p}(\mathrm{F}{=}\text{catfish}, \mathrm{L}{=}16, \mathrm{W}{=}2.1)}$$

We can calculate the probability of all three possibilities (which will add up to 1.0).

## 3.2 Naive Bayes

The assumption in naive Bayes is that the probability distributions of length and weight are conditionally independent given the breed of the fish. That is:

$$p(\text{F=trout}, \text{L=16}, \text{W=2.1}) = p(\text{L=16}|\text{F=trout})p(\text{W=2.1}|\text{F=trout})P(\text{F=trout})$$

Within a particular species, things like length and weight, which represent the sum of a lot of genetic and environmental factors, are often normally distributed.

What if we assume that length has a normal distribution for each kind of fish? What is the most likely estimator for the mean of each trait?

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

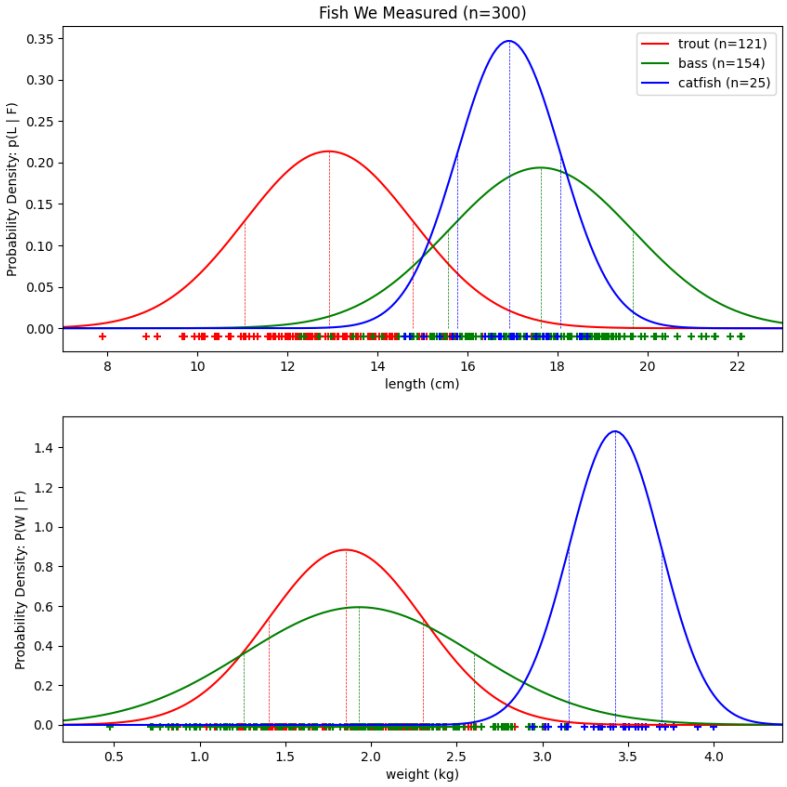
And variance?

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n} (\mu - x_i)^2$$

We can compute those pretty easily:

| Fish | Trait | $\mu$ | $\sigma^2$ |
|---|---|---|---|
| Trout | Length | 12.91522571 | 3.49054363 |
| | Weight | 1.85290281 | 0.20387574 |
| Bass | Length | 17.62628327 | 4.23881747 |
| | Weight | 1.92960088 | 0.45092986 |
| Catfish | Length | 16.92187387 | 1.32301935 |
| | Weight | 3.42423185 | 0.07244546 |

If we plot those probability distributions out:




(Source:`2_univariates.py`)

Note that the area under each one of those curves is exactly 1.0. They are probability distributions.

Now we can use the formula for the normal distribution:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Thus,

| Given | $p(L = 16\|F)$ | $p(W = 2.1\|F)$ |
|---|---|---|
| F=Trout | .0546 | .7607 |
| F=Bass | .1418 | .5753 |
| F=Catfish | .2516 | 0.01 |

Using the naive assumption, we can now compute the joint probability for any type of fish:

$$p(F = ?, L = 16, W = 2.1) = p(W = 2.1|F = ?)p(L = 16|F = ?)p(F = ?)$$

| For | $p(L = 16, W = 2.1, F = ?)$ |
|---|---|
| F=Trout | .016763 |
| F=Bass | .041886 |
| F=Catfish | 0.00001 |

The sum of these is $p(L = 16, W = 2.1)$: .0586.

By the definition of conditional probability:

$$p(F = ?|L = 16, W = 2.1) = \frac{p(F = ?, L = 16, W = 2.1)}{p(L = 16, W = 2.1)}$$

So,

| For | $p(F = ?|L = 16, W = 2.1)$ |
|---|---|
| F=Trout | .286 |
| F=Bass | .714 |
| F=Catfish | $\approx 0$ |

If you pull a random fish out of the lake and its length is 16 and its weight is 2.1, there is a 71.4% chance that it is bass. There is a 28.6% chance it is a trout.

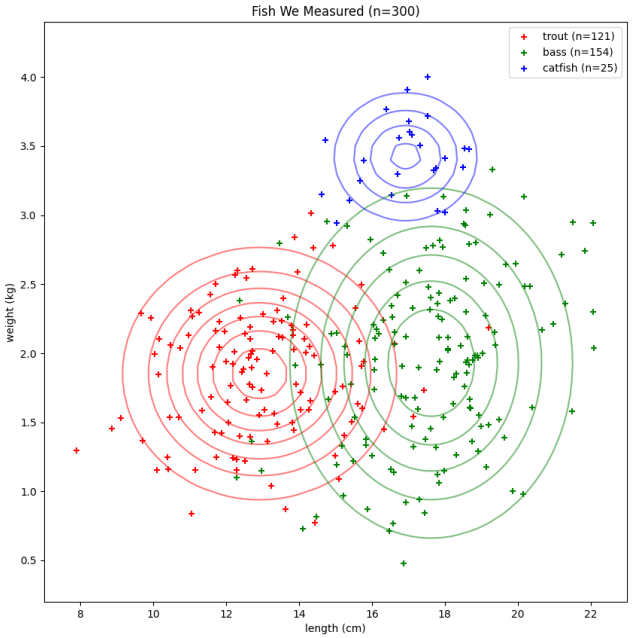
## 3.3 Using the multivariate Gaussian distribution

Given the kind of fish, length and width are independent!? That makes no sense. Longer fish tend to be heavier, right?

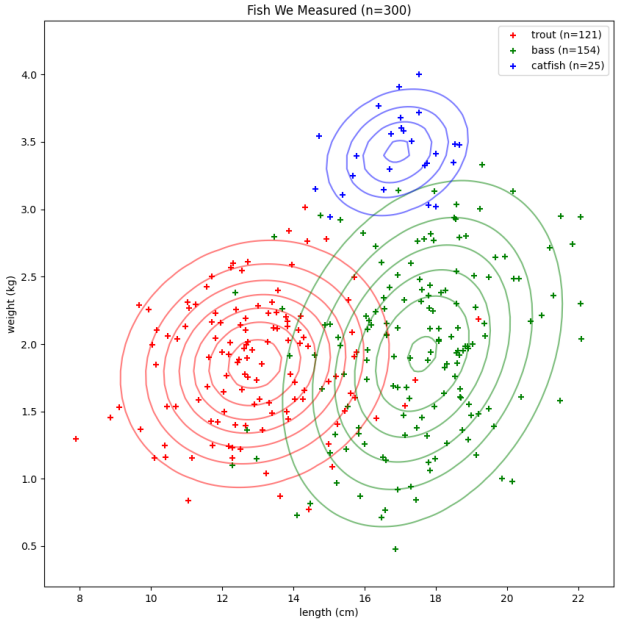
The multivariate Gaussian distribution is a generalization of the normal distribution::

- It deals with data of any dimension d.
- It can include the idea of covariance. For example, length and width are not independent.

If we use your naive Bayes, we can get two-dimensional density plots that would look like this:




With independence, there are ovals that are circles, ovals that go up, or ovals that sideways. If we use the multivariate Gaussian distribution, we get:




When there is covariance, the ovals can slant in any direction.

If we discard the naive assumption, we should get slightly more accurate results.

The multivariate Gaussian distribution uses vectors and matrices. The mean $\vec{\mu}$ is a vector

of dimension d. The variance of each variable and its covariance with the other variables is captured in a $d \times d$ matrix called *the covariance matrix* is usually called $\mathbf{\Sigma}$. (Yes, this is also the symbol for summation. This creates some confusion at times, but you can usually figure out which is intended by the context.)

When you know $\vec{\mu}$ and $\mathbf{\Sigma}$, the probability density for any vector $\vec{x}$ is:

$$p(\vec{x}) = (2\pi)^{-d/2} \det(\mathbf{\Sigma})^{-1/2} \exp\left(-\frac{1}{2}(\vec{x} - \vec{\mu})^T \mathbf{\Sigma}^{-1} (\vec{x} - \vec{\mu})\right)$$

If you have several data points $\vec{x_1}, ..., \vec{x_n}$, how do you compute the values of $\vec{\mu}$ and $\mathbf{\Sigma}$ for which the observations $\vec{x_1}, ..., \vec{x_n}$ would be most likely?

$$\vec{\mu} = \frac{1}{n} \sum_{i=1}^{d} \vec{x_i}$$

And the covariance matrix? (Here, we think of each $\vec{x_i}$ and $\vec{\mu}$ as column vectors.)

$$\mathbf{\Sigma} = \frac{1}{n} \sum_{i=1}^{n} (\vec{x_i} - \vec{\mu})(\vec{x_i} - \vec{\mu})^T$$

Notice that the diagonal entries of the matrix are the variance of each variable. Notice also that this is a symmetric matrix.

Applying this to our data we get:

| Fish | $\vec{\mu}$ | $\mathbf{\Sigma}$ |
|---|---|---|
| Trout | $\begin{bmatrix} 12.91522571 \\ 1.85290281 \end{bmatrix}$ | $\begin{bmatrix} 3.51963149 & 0.09975744 \\ 0.09975744, & 0.20557471 \end{bmatrix}$ |
| Bass | $\begin{bmatrix} 17.62628327 \\ 1.92960088 \end{bmatrix}$ | $\begin{bmatrix} 4.26652216 & 0.39553268 \\ 0.39553268 & 0.45387711 \end{bmatrix}$ |
| Catfish | $\begin{bmatrix} 16.92187387 \\ 3.42423185 \end{bmatrix}$ | $\begin{bmatrix} 1.37814515 & 0.07281453 \\ 0.07281453 & 0.07546403 \end{bmatrix}$ |

We can use these to compute the likelihood of seeing a fish with length = 16 and weight = 2.1 for each type of fish:

| Given | $p(L = 16, W = 2.1 \mid F = ?)$ |
|---|---|
| F=Trout | .04578 |
| F=Bass | .07732 |
| F=Catfish | .000001 |

Multiplied by the prior to get the joint probability:

| Given | $p(L = 16, W = 2.1, F = ?)$ |
|---|---|
| F=Trout | .01847 |
| F=Bass | .03969 |
| F=Catfish | $\approx 0$ |

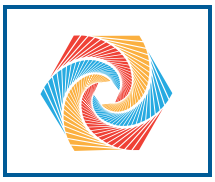
We sum those to get $p(L = 16, W = 2.1) = .0582$.

| Given | $p(F = ? \mid L = 16, W = 2.1)$ |
|---|---|
| F=Trout | .318 |
| F=Bass | .682 |
| F=Catfish | $\approx 0$ |

Given a randomly selected fish that is 19 inches long and 2.1 kg in weight, you would guess that it is a bass with a confidence of 68.2%.

Appendix A

# Answers to Exercises



# Index

covariance, 5

Gaussian distribution, 5

multivariate normal, 5
multivariate normal distribution, 5

numerical integration, 10